

Week 1, Days 1–2

Attention, RoPE, and MHA → MQA → GQA, derived on this repository's measured configuration

Running example throughout: **Qwen/Qwen2.5-1.5B, fp16, one NVIDIA RTX 3060 12GB (Ampere, sm_86)**. Every architectural claim terminates in a number that is either measured in `llm-inference-optimization` or derivable by hand from its config. Part 0 assumes nothing and defines every term Part 1 uses. Part 1 is the actual Days 1–2 material.

CONFIG VERIFICATION – WHAT WAS CHECKED BEFORE WRITING

The config carried in from memory was `d_model 1536`, 28 layers, 12 query heads, 2 KV heads, `head_dim 128`, vocab 151,936, intermediate 8960, params 1,543,714,304, weights ≈ 2944 MiB. **All of it holds.** Layers, head counts and `head_dim` appear in five documents and in `scripts/oom_sweep.py`; `d_model = 1536` is confirmed independently by `docs/baseline-hf-results.md` ("the prefill GEMMs are M=512, K=1536") and by the identity `12 × 128 = 1536`; and `intermediate = 8960` and `vocab = 151,936` are confirmed by the parameter sum in **WORKED EXAMPLE 1-A**, which reproduces 1,543,714,304 exactly. Two clarifications rather than corrections: the fp16 *floor* is **2944.40 MiB** and the *measured resident* figure in `results/baseline_hf.csv` is **2945.29 MiB**, 0.89 MiB above it; and the head-count notation matters — 12 and 2 are query and KV heads respectively, and it is the **2** that sizes the cache.

PART 0 — Fundamentals

0.1 What a language model actually does

A language model is a function from a sequence of token ids to a probability distribution over the next token:

$$p(x_t \mid x_1 \dots x_{t-1}) \quad \text{modelled as} \quad \text{softmax}(f(x_1 \dots x_{t-1})) \quad \text{over } V \text{ possible tokens}$$

Text is produced by running that function in a loop: sample or argmax a token, append it to the sequence, run again. Two consequences fall straight out of the loop, and the whole field lives inside them.

1. **Generation is inherently sequential.** Token `t+1` cannot be computed until token `t` exists, because it is part of the input. No amount of hardware parallelises across the time axis of a single sequence. This is why single-stream decode throughput is so much lower than the same model's raw compute capability would suggest.
2. **Each step re-runs over a sequence one token longer.** Naively, step `t` recomputes everything steps `1..t-1` already computed. Total work would be $O(n^2)$ forward passes for `n` tokens.

Consequence 2 is **the reason the KV cache exists**. Almost every intermediate in a transformer depends only on one token's own representation, and can therefore be discarded. The exceptions are the keys and values in attention: token `t`'s query must attend to *every* earlier token's key and value. So you store those, and only those, and each new step contributes exactly one new key and one new value per layer per KV head. That converts the quadratic recomputation into a linear-in-context *read*, and turns a compute problem into a memory problem. Everything downstream — PagedAttention, GQA, KV quantization, the OOM experiment in this repo — is a consequence of that trade.

0.2 Tokens

Models do not see characters. A tokenizer (BPE or a variant) maps text to integer ids drawn from a fixed vocabulary, here of size **151,936**. The ids carry no meaning — id 5000 is not "between" 4999 and 5001. They are

indices into a lookup table, and the table is where meaning lives.

Rule of thumb for English: ~4 characters per token, or ~0.75 tokens per word. Useful because every context-length and cost number in the field is quoted in tokens and every user-facing quantity is in words. The 123,565-token OOM point in this repo is therefore roughly 90,000 words, or a long novel.

0.3 Embeddings

The embedding is a matrix of shape `[V, d_model]`. Token id `i` selects row `i`. That is the entire operation: no multiply, no add, one strided read.

WORKED EXAMPLE 0-A – WHAT THE EMBEDDING TABLE COSTS

shape	<code>[V, d_model] = [151,936, 1536]</code>		
parameters	<code>151,936 x 1,536</code>	<code>= 233,373,696</code>	
fp16 bytes	<code>233,373,696 x 2</code>	<code>= 466,747,392 B</code>	
	<code>466,747,392 / 1024^2</code>	<code>= 445.13 MiB</code>	
fraction of the model			
	<code>233,373,696 / 1,543,714,304</code>	<code>= 15.12%</code>	

Qwen2.5-1.5B *ties* its input embedding and output projection, so this table is counted once and serves both. Untied, it would appear twice and the model would be 1.777B parameters instead of 1.544B.

The point to carry forward: 15% of this model is a structure that performs *zero arithmetic*. It is pure memory traffic. That is the first instance of the theme of the entire field — that on modern hardware the cost of an operation is usually the cost of moving its operands, not the cost of the operation.

0.4 The dot product, and why attention uses it

$a \cdot b = \sum_i a_i b_i = |a| |b| \cos \theta$. For unit vectors it is the cosine, so it is a similarity score bounded in $[-1, 1]$.

WORKED EXAMPLE 0-B – THE DOT PRODUCT AS ALIGNMENT

<code>a = (1, 0)</code>	all unit length			
<code>b = (1, 0)</code>	<code>theta = 0 deg</code>	<code>a.b = 1*1 + 0*0</code>	<code>= 1.000</code>	identical
<code>b = (0.7071, 0.7071)</code>	<code>theta = 45 deg</code>	<code>a.b = 1*0.7071 + 0*0.7071</code>	<code>= 0.707</code>	similar
<code>b = (0, 1)</code>	<code>theta = 90 deg</code>	<code>a.b = 1*0 + 0*1</code>	<code>= 0.000</code>	unrelated
<code>b = (-0.7071, 0.7071)</code>	<code>theta = 135 deg</code>	<code>a.b = -0.7071</code>	<code>= -0.707</code>	dissimilar
<code>b = (-1, 0)</code>	<code>theta = 180 deg</code>	<code>a.b = -1</code>	<code>= -1.000</code>	opposite

Attention needs "how relevant is token `j` to token `i`", and it needs it for every pair, cheaply. A dot product is a single fused-multiply-add chain per pair and maps directly onto a matrix multiply, which is the one operation GPUs are built to do. That is the whole justification: it is a similarity measure that happens to be a GEMM.

0.5 Matrix multiplication, GEMM vs GEMV

$[M,K] \times [K,N] \rightarrow [M,N]$. Output element `C[i][j]` is the dot product of row `i` of A with column `j` of B. The inner dimension `K` must match and is contracted away.

WORKED EXAMPLE 0-C – A [2,3] X [3,2] BY HAND

```
A = [ 1  2  3 ]      B = [  7  8 ]
    [ 4  5  6 ]      [  9 10 ]
                        [ 11 12 ]

C[0][0] = 1*7 + 2*9 + 3*11 = 7 + 18 + 33 = 58
C[0][1] = 1*8 + 2*10 + 3*12 = 8 + 20 + 36 = 64
C[1][0] = 4*7 + 5*9 + 6*11 = 28 + 45 + 66 = 139
C[1][1] = 4*8 + 5*10 + 6*12 = 32 + 50 + 72 = 154

C = [ 58  64 ]      shape [2,2] = [M,N]
    [139 154 ]

FLOPs = 2*M*K*N = 2*2*3*2 = 24      (12 multiplies + 12 adds; check: 4 outputs x 3 MACs x 2)
```

GEMM VS GEMV – THE ENTIRE PERFORMANCE STORY REDUCES TO WHICH ONE YOU ARE RUNNING

	GEMV (M = 1)	GEMM (M large)
What it is	matrix × <i>vector</i>	matrix × matrix
Weight bytes read	K×N×2	K×N×2 — <i>the same</i>
FLOPs	2KN	2MKN
Arithmetic intensity	2KN / 2KN = 1 FLOP/byte	≈ M FLOP/byte
Where it occurs	decode , batch 1	prefill , and batched decode
Bound by	memory bandwidth	compute (once M is large enough)

Read the second row again. A GEMV reads exactly as many weight bytes as a GEMM and does 1/M of the arithmetic with them. That single asymmetry is why batch-1 decode is slow, why batching helps, why prefill is fast, and why this repo's batching sweep exists.

0.6 Linear layers, and why weight memory is a constant

$y = xW + b$, with W of shape $[d_{in}, d_{out}]$. The critical property for inference is that **W is fixed and shared across every token, every request, and every batch element**. The same 2945 MiB is read whether you are serving one sequence or two hundred.

That constancy is precisely why batching works. If weight bytes were per-token, batching would buy nothing: doubling the tokens would double the traffic. Because they are per-*step*, doubling the batch doubles the useful output for the same weight read. The batching sweep in this repo measures exactly this and finds it is worth 30× before it runs out.

0.7 Counting FLOPs, and the "2 × params per token" rule

A $[M,K] \times [K,N]$ GEMM costs $2 \cdot M \cdot K \cdot N$ FLOPs: each of the $M \cdot N$ outputs is a sum over K terms, and each term is one multiply plus one add.

WORKED EXAMPLE 0-D – DERIVING 2 X PARAMS PER TOKEN ON ONE REAL PROJECTION

```

take gate_proj from one layer of this model: W has shape [1536, 8960]
parameters in W           = 1,536 x 8,960           = 13,762,560

prefill over 512 tokens: [512, 1536] x [1536, 8960]
FLOPs                     = 2 x 512 x 1,536 x 8,960   = 14,092,861,440

per token                  = 14,092,861,440 / 512       = 27,525,120

RATIO                      = 27,525,120 / 13,762,560   = 2.0000 exactly

```

Why it is exactly 2, and not approximately 2. Producing one token's output requires touching every weight in W exactly once, and each touch is one multiply and one add. Two floating-point operations per parameter. There is no slack in the argument, which is why the rule is quoted as an equality rather than an estimate.

The caveat you must state when you use it. The rule covers linear layers only. It ignores the attention score-and-weight term, which costs $O(\text{seq}^2)$ in prefill while the linear term costs $O(\text{seq})$. At a 512-token prompt attention adds only ~3% on top; past a few thousand tokens the linear-only estimate undercounts badly. It also ignores softmax, normalisation and elementwise ops, which are cheap in FLOPs but *not* cheap in memory traffic — which is exactly why fusing them is worth anything.

```

whole-model form: FLOPs per token ~ 2 x 1,543,714,304 = 3.087 GFLOP
prefill of 512: 2 x 1.5437e9 x 512 = 1.581 TFLOP

```

0.8 Nonlinearity: SiLU and SwiGLU

Stacked linear layers collapse: $(xW_1)W_2 = x(W_1W_2)$, so any depth of pure linear layers is representationally a single linear layer. A nonlinearity between them is what makes depth mean anything.

$\text{SiLU}(x) = x \cdot \sigma(x)$, where $\sigma(x) = 1/(1+e^{-x})$.

WORKED EXAMPLE 0-E – SILU EVALUATED

x	sigma(x)	SiLU(x) = x*sigma(x)	
-4	0.017986	-0.0719	near zero, but not exactly zero (unlike ReLU)
-2	0.119203	-0.2384	
-1	0.268941	-0.2689	the minimum region: SiLU is non-monotonic here
0	0.500000	0.0000	
1	0.731059	0.7311	
2	0.880797	1.7616	
4	0.982014	3.9281	approaches x for large x

Smooth everywhere, unbounded above, bounded below, and slightly negative for small negative inputs, which lets a unit express "mildly suppress" instead of only "off".

SwiGLU is the gated three-matrix form this model uses:

```

SwiGLU(x) = ( SiLU(x @ W_gate) * (x @ W_up) ) @ W_down      * is elementwise

shapes for this model:  W_gate [1536, 8960]    W_up [1536, 8960]    W_down [8960, 1536]
parameters:           13,762,560 x 3 = 41,287,680 per layer

```

Three matrices instead of two, so for a fixed parameter budget the intermediate is narrower than a plain FFN's $4 \times d$ would be: $8960 / 1536 = 5.83\times$ across three matrices rather than $4\times$ across two. The gate branch lets the layer modulate the up branch multiplicatively, which is a strictly richer operation than a pointwise nonlinearity alone.

0.9 RMSNorm

$$\text{RMSNorm}(x)_i = x_i / \sqrt{(1/d) * \sum_j x_j^2 + \text{eps}} * g_i$$

where g is a learned per-channel gain of shape $[d_{\text{model}}]$ — 1536 parameters, two per layer in this model, 1536 more for the final norm.

WORKED EXAMPLE 0-F — RMSNORM ON A SMALL VECTOR, THEN VERIFIED

$x = (3, -1, 4, -2)$, $d = 4$, gain $g = 1$ (for clarity), $\text{eps} = 1e-6$ (negligible here)

sum of squares	= $9 + 1 + 16 + 4$	= 30
mean square	= $30 / 4$	= 7.5
RMS	= $\sqrt{7.5}$	= 2.738613

$y_1 = 3 / 2.738613 = 1.095445$
 $y_2 = -1 / 2.738613 = -0.365148$
 $y_3 = 4 / 2.738613 = 1.460593$
 $y_4 = -2 / 2.738613 = -0.730297$

VERIFY -- recompute the RMS of the output:

squares	= $1.200000 + 0.133333 + 2.133333 + 0.533333$	= 4.000000
mean square	= $4.0 / 4$	= 1.000000
RMS	= $\sqrt{1.0}$	= 1.000000 <-- exactly 1, as designed

Against LayerNorm. LayerNorm subtracts the mean first: $(x - \mu)/\sigma \cdot g + b$. RMSNorm drops the mean subtraction and the bias. The empirical finding (Zhang & Sennrich, 2019) is that the re-centering contributes essentially nothing to quality in these architectures while the re-scaling does all the work. Modern models dropped it for the systems reason: mean subtraction is a second reduction pass over the vector, so removing it halves the reduction traffic in a kernel that is memory-bound to begin with. In this repo's traces, fused RMSNorm runs **56 times per decode step** at $\sim 3.5\text{--}3.9\ \mu\text{s}$ each — individually trivial, collectively $\sim 200\ \mu\text{s}$, and 56 more launches if it is *not* fused. That is why every serving engine fuses it.

0.10 Residual connections and the residual stream

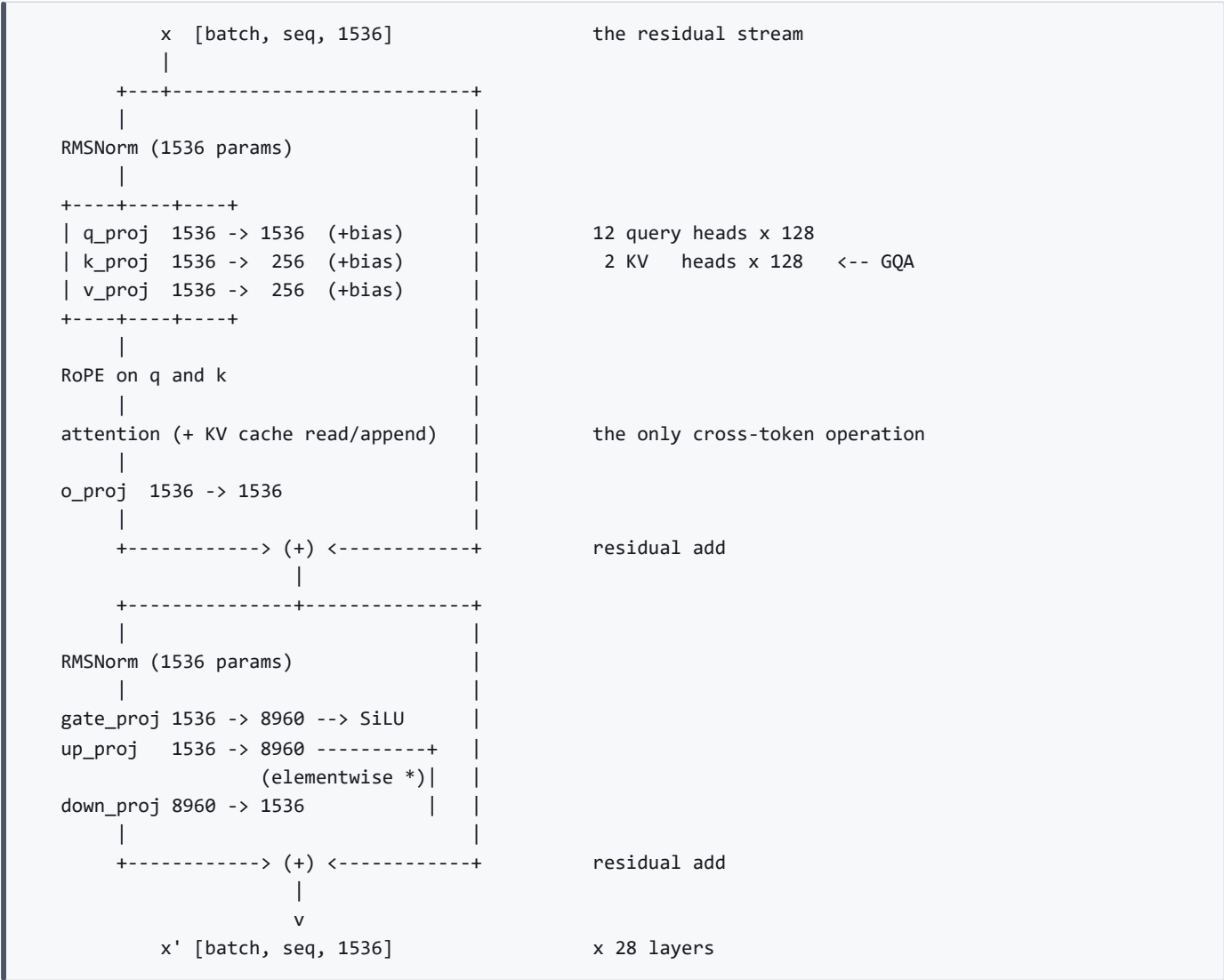
```
x = x + Attention(RMSNorm(x))
x = x + MLP(RMSNorm(x))
```

pre-norm, which is what this model uses

Two readings, both correct and both worth having.

- **The gradient-path reading.** The identity branch gives every layer a direct route to the loss, so gradients do not have to survive multiplication by 28 Jacobians. This is what made depth trainable.
- **The interpretability reading.** The residual is not a "skip" — it is the main object. Think of a single running representation, the *residual stream*, of width $d_{\text{model}} = 1536$, that every block reads from and writes an increment back into. Blocks communicate through it. That is why d_{model} is fixed for the whole depth of the network while the FFN intermediate is free to be 8960.

o.11 The transformer block assembled



THE ONE-LINE DIVISION OF LABOUR

Attention moves information *between* tokens. The MLP processes each token independently. The MLP never looks sideways; run it on a batch of tokens and each row is computed in complete isolation from the others. Every cross-token dependency in the entire architecture lives in one operation.

WORKED EXAMPLE 0-G — THE PER-LAYER PARAMETER SPLIT, AND ITS CONSEQUENCE

ATTENTION

q_proj	1,536 x 1,536 + 1,536 bias	=	2,360,832	
k_proj	1,536 x 256 + 256 bias	=	393,472	
v_proj	1,536 x 256 + 256 bias	=	393,472	
o_proj	1,536 x 1,536, no bias	=	2,359,296	
	subtotal	=	5,507,072	11.8%

MLP

gate_proj	1,536 x 8,960	=	13,762,560	
up_proj	1,536 x 8,960	=	13,762,560	
down_proj	8,960 x 1,536	=	13,762,560	
	subtotal	=	41,287,680	88.2%

NORMS	2 x 1,536	=	3,072	0.007%
PER LAYER		=	46,797,824	

Draw the consequence out loud, because it is the sentence that organises the whole field: *most of the weight traffic is MLP, and almost all of the systems complexity is attention.* The MLP is 88% of the bytes a decode step must read, and it is three dense GEMMs that cuBLAS already runs near peak — there is nothing to invent. Attention is 12% of the bytes and generates the KV cache, its paging, its fragmentation, FlashAttention, GQA, split-KV decoding, and every OOM in this repo. Optimisation effort goes where the *difficulty* is, not where the bytes are.

0.12 Number formats

Format	Bytes	Layout (sign / exp / mantissa)	Note
fp32	4	1 / 8 / 23	the reference; rarely used for inference weights
fp16	2	1 / 5 / 10	this repo's target. Max finite value 65,504
bf16	2	1 / 8 / 7	fp32's exponent range, 3 fewer mantissa bits
int8	1	integer + a scale	needs per-tensor or per-channel scales
NF4	0.5	4-bit normal-float + block scales	the quantization run in this repo

fp16 vs bf16 in one sentence: same 2 bytes, but bf16 trades mantissa precision for fp32's exponent range, so it rarely overflows and rarely needs loss scaling, while fp16 has finer resolution near 1.0 but a ceiling of 65,504 that real activations can hit. *This repo deliberately targets fp16, never bf16* — the 3060 supports bf16, but fp16 keeps the numbers comparable with later V100/T4 runs that do not, and it changes nothing in the KV-cache arithmetic since both are 2 bytes per element.

WORKED EXAMPLE 0-H – THE MB / MiB TRAP, ON THIS MODEL'S WEIGHTS

```
weight bytes = 1,543,714,304 params x 2 bytes = 3,087,428,608 B

binary units (what torch, nvidia-smi and this repo report)
  / 1024^2 = 2,944.40 MiB
  / 1024^3 =    2.875 GiB

decimal units (what marketing, some papers and most blog posts report)
  / 1e6     = 3,087.43 MB
  / 1e9     =    3.087 GB

THE GAP
MB vs MiB   3,087.43 / 2,944.40 = 1.0486 -> 4.86%
GB vs GiB   3.087 / 2.875 = 1.0737 -> 7.37%
```

A 7% discrepancy is exactly the size that looks like a measurement bug and sends you chasing a phantom. It is also small enough to survive unnoticed into a comparison table, where it silently favours whichever side used decimal. **The convention this repo uses, and the one to state before quoting any figure:** VRAM is always MiB (binary), because that is what `torch.cuda.memory_allocated()` and `nvidia-smi` report; bandwidth is always GB/s (decimal), because that is how memory bus specifications are defined. So "3.09 GB of weights read at 360 GB/s" and "2945 MiB resident" describe the same weights in two unit systems, on purpose.

0.13 What a GPU is, in the three terms that matter

Term	What it means	RTX 3060 12GB (sm_86)
Compute throughput	FLOPs/s the tensor cores can retire on dense fp16	~51 TFLOPS (fp32 accum) ~100 TFLOPS (fp16 accum)
Memory bandwidth	bytes/s between HBM/GDDR and the SMs. The number that decides decode.	360.0 GB/s theoretical 291.5 GB/s achievable
Memory capacity	how much must fit at once: weights + KV cache + activations	12,288 MiB

WORKED EXAMPLE 0-I — WHERE 360 GB/S COMES FROM, AND WHY THE HONEST CEILING IS 291.5

THEORETICAL, from what NVML says the card is actually set to (not from the model name):

memory clock	7,501 MHz
GDDR6 transfers on both edges	x 2
bus width	192 bits
-> $7.501e9 \times 2 \times 192 / 8 \text{ bytes}$	= 360.0 GB/s

ACHIEVABLE, measured by a large streaming read (scripts/measure_bandwidth.py):
291.5 GB/s = 81% of theoretical

81% is the normal range; no kernel exceeds the achievable figure, so it is the honest denominator for any "% of peak" claim. The distinction is not pedantry — it changes conclusions. This repo's vLLM decode step reads **68% of theoretical**, which sounds like a third of the performance is still on the table, or **85% of achievable**, which says the batch-1 scheduling problem is essentially solved and the next win must come from somewhere else entirely. Same measurement, opposite strategic conclusion.

The corollary, learned the hard way in this repo: two rented boxes with the same GPU name and the same VRAM ran identical work at a uniform **1.22×** difference (spread 1.4% over four runs). The obvious memory-downclock explanation was tested against NVML and killed — both cards were at stock 7501 MHz. The cause is still unexplained. Never compare timings across machines.

0.14 Compute-bound vs memory-bound — the most important result in Part 0

Arithmetic intensity is FLOPs performed per byte moved from memory. Every operation has one. The hardware has a **ridge point** where its two ceilings cross:

$\text{ridge} = \text{peak FLOPS} / \text{peak bandwidth} = 51e12 / 360e9 = 141.7 \text{ FLOP/byte}$

intensity < ridge	-> MEMORY-BOUND	the bus is saturated, the ALUs wait
intensity > ridge	-> COMPUTE-BOUND	the ALUs are saturated, the bus waits

WORKED EXAMPLE 0-J – WHERE DECODE AND PREFILL SIT ON THIS CARD

DECODE, batch 1, one token

FLOPs $\sim 2 \times \text{params} = 2 \times 1,543,714,304 = 3.087\text{e}9$
 bytes $\sim 1 \times \text{params} \times 2 \text{ bytes} = 3.087\text{e}9$ (each weight read once)
 intensity $= 3.087\text{e}9 / 3.087\text{e}9 = 2.0 \text{ FLOP/byte}$

Note it is exactly 2.0, and the reason is pretty: fp16 weights are 2 bytes each and each weight is worth 2 FLOPs (one multiply, one add). The units cancel.

ridge is 141.7. Decode sits 71x to the LEFT of it.

PREFILL, 512 tokens

FLOPs $= 2 \times \text{params} \times 512 = 1.581\text{e}12$
 bytes $\sim 1 \times \text{params} \times 2 \text{ bytes} = 3.087\text{e}9$ (the SAME weight read, amortised)
 intensity $= 1.581\text{e}12 / 3.087\text{e}9 = 512 \text{ FLOP/byte}$

ridge is 141.7. Prefill sits 3.6x to the RIGHT of it.

This is the single most consequential fact in LLM inference, and it is worth saying in the strongest form: no kernel, compiler, or engine can move batch-1 decode off the memory wall.

At intensity 2 against a ridge of 142, you are not 5% short of compute-bound; you are 71x short. The only things that can help are (a) moving fewer bytes — quantization, KV compression — or (b) doing more work per byte moved — batching, speculative decoding. Faster arithmetic is irrelevant. Every optimization in this repo is one of those two, and knowing which is which is most of the subject.

0.15 Prefill vs decode

	Prefill	Decode
What it does	one forward pass over the whole prompt	one forward pass per generated token
Parallelism	all P prompt positions at once	one position; strictly sequential
Weight op shape	GEMM, M = 512	GEMV, M = 1
Arithmetic intensity	$\sim 512 \text{ FLOP/byte}$	$\sim 2 \text{ FLOP/byte}$
Bound by	compute	memory bandwidth
KV cache	<i>writes</i> P tokens of it	<i>reads</i> all of it, <i>appends</i> one token
Latency metric	TTFT (time to first token)	TPOT / ITL (inter-token latency)
Measured here (HF)	122 ms $\rightarrow$ 4205 tok/s	28.1 tok/s

That is a **150x** throughput gap on the same model, the same card, the same weights, in the same process. Nothing about the arithmetic changed; only the shape of the operation did. Prefill amortises one weight read across 512 positions of parallel work. Decode pays a full weight read for one token.

o.16 Vocabulary checkpoint

Term	Definition, with this model's value where one exists
d_model	width of the residual stream; fixed for the whole depth. 1536
head_dim	width of one attention head. 128 ; $12 \times 128 = 1536$
Query / KV heads	12 / 2 . Under GQA these differ; the KV count sizes the cache
Residual stream	the running [. . , . . , 1536] representation every block reads and adds into
GEMM / GEMV	matrix×matrix / matrix×vector. Same weight bytes, M × different FLOPs
Arithmetic intensity	FLOPs per byte moved. Decode ~2; prefill ~512; ridge 142
Ridge point	peak FLOPS / peak bandwidth. 141.7 FLOP/byte here
Prefill / decode	the parallel prompt pass / the sequential per-token loop
KV cache	stored keys and values for all past tokens. 28 KiB per token here
MBU / MFU	fraction of peak bandwidth / peak FLOPS actually achieved
TTFT / TPOT	time to first token / time per output token
allocated / reserved	bytes held by live tensors / bytes PyTorch's caching allocator holds from the driver
Achievable bandwidth	what a streaming read really reaches. 291.5 GB/s , 81% of theoretical

PART 1 — Attention, RoPE, and GQA

1.1 The model we will compute with

Field	Value	Field	Value
hidden size <code>d_model</code>	1,536	vocab	151,936
layers	28	intermediate	8,960
query heads	12	tied embeddings	yes
KV heads	2	dtype	fp16 (2 B)
head_dim	128	GQA group size	6
parameters	1,543,714,304	weights (fp16 floor)	2,944.40 MiB

WORKED EXAMPLE 1-A — THE PARAMETER COUNT, SUMMED TO THE PUBLISHED TOTAL

```
EMBEDDING (tied: counted once, serves both input lookup and output projection)
151,936 x 1,536 = 233,373,696

PER LAYER
q_proj 1,536 x 1,536 + 1,536 bias = 2,360,832
k_proj 1,536 x 256 + 256 bias = 393,472 <-- 2 KV heads x 128 = 256
v_proj 1,536 x 256 + 256 bias = 393,472
o_proj 1,536 x 1,536 (no bias) = 2,359,296
attention subtotal = 5,507,072
gate_proj 1,536 x 8,960 = 13,762,560
up_proj 1,536 x 8,960 = 13,762,560
down_proj 8,960 x 1,536 = 13,762,560
MLP subtotal = 41,287,680
input_layernorm + post_attention_layernorm 2 x 1,536 = 3,072
PER LAYER = 46,797,824

TOTAL
46,797,824 x 28 layers = 1,310,339,072
+ embedding = 233,373,696
+ final RMSNorm = 1,536
TOTAL = 1,543,714,304 <-- exact match

fp16 bytes 1,543,714,304 x 2 = 3,087,428,608 B = 2,944.40 MiB
measured results/baseline_hf.csv weights_vram_mib = 2,945.29 MiB
excess 0.89 MiB (fp32 inv_freq buffers, ~7 KB, plus allocator segment rounding)
```

Why this exercise is worth doing rather than looking up. Being able to sum a model to its published parameter count is a standard interview task, and the sanity check it enables is a real one. In this repo, transformers 4.44.2 reported **4737 MiB** resident weights against this 2944 MiB floor. The 1.79 GB excess was not weights at all: it was per-layer RoPE `cos_cached` / `sin_cached` buffers sized to the full 131,072-token context — $28 \times 2 \times 131,072 \times 128 \times 2 \text{ B} = 1,792 \text{ MiB}$, matching the excess exactly. Upgrading to 4.46.3, which hoists rotary to one model-level module, dropped resident weights to 2945 MiB. **That 1.79 GB was worth roughly 64,000 tokens of context in the OOM experiment.** The only reason it was caught is that someone knew what the floor should be.

1.2 What attention is actually for

The MLP processes each token in isolation, so on its own a stack of MLPs is 28 independent per-token functions and the sequence structure is invisible to it. Something has to let a token's representation depend on other tokens. That something must be able to decide *which* other tokens matter, differently for each token, at inference time, over sequences of any length — because "which earlier word does *it* refer to" is a different question at every position.

Attention is a differentiable, content-addressed lookup. Each token emits a **query** ("what am I looking for"), every token emits a **key** ("what I can be found by") and a **value** ("what I will contribute if found"). Relevance is the query·key dot product, softmax turns those scores into weights summing to 1, and the output is that weighted average of values. Nothing about it is hard-coded to language; it is a soft dictionary lookup where the match is a similarity rather than an equality.

1.3 Q, K, V

$q = x @ W_q + b_q$	$[.., .., 1536] \rightarrow [.., .., 1536]$	12 query heads x 128
$k = x @ W_k + b_k$	$[.., .., 1536] \rightarrow [.., .., 256]$	2 KV heads x 128
$v = x @ W_v + b_v$	$[.., .., 1536] \rightarrow [.., .., 256]$	2 KV heads x 128

All three are linear projections of the *same* input vector. The asymmetry in output width is GQA and nothing else: 12 query heads, 2 KV heads, so each KV head is shared by 6 query heads. Note immediately that **k** and **v** are **6× narrower than q** — that ratio is the entire KV-cache saving, and §1.10 quantifies it.

1.4 The attention formula, worked fully by hand

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_{\text{head}}}} + \text{mask}\right) V$$

WORKED EXAMPLE 1-B — 3 TOKENS, D_HEAD = 4, ONE HEAD, CAUSAL

```

Q = [ 1 0 1 0 ]   K = [ 1 0 1 0 ]   V = [ 1 2 3 4 ]
    [ 0 1 0 1 ]       [ 0 1 1 0 ]       [ 5 6 7 8 ]
    [ 1 1 0 0 ]       [ 1 1 0 1 ]       [ 9 10 11 12 ]

STEP 1 -- Q K^T      [3,4] x [4,3] -> [3,3]
S[0][0] = 1*1+0*0+1*1+0*0 = 2      S[0][1] = 1*0+0*1+1*1+0*0 = 1      S[0][2] = 1*1+0*1+1*0+0*1 = 1
S[1][0] = 0                        S[1][1] = 1                        S[1][2] = 0*1+1*1+0*0+1*1 = 2
S[2][0] = 1*1+1*0+0*1+0*0 = 1      S[2][1] = 1*0+1*1+0*1+0*0 = 1      S[2][2] = 1*1+1*1+0*0+0*1 = 2

S = [ 2 1 1 ]
    [ 0 1 2 ]
    [ 1 1 2 ]

STEP 2 -- scale by 1/sqrt(d_head) = 1/sqrt(4) = 0.5
S' = [ 1.0  0.5  0.5 ]
     [ 0.0  0.5  1.0 ]
     [ 0.5  0.5  1.0 ]

STEP 3 -- causal mask: position i may attend only to j <= i. Set j > i to -inf.
S'' = [ 1.0 -inf -inf ]
      [ 0.0  0.5 -inf ]
      [ 0.5  0.5  1.0 ]

STEP 4 -- row-wise softmax
row 0: exp(1.0)=2.718282          sum=2.718282 -> [1.000000, 0, 0]
row 1: exp(0.0)=1.000000 exp(0.5)=1.648721 sum=2.648721 -> [0.377541, 0.622459, 0]
row 2: exp(0.5)=1.648721 exp(0.5)=1.648721 exp(1.0)=2.718282
      sum=6.015724
      -> [0.274069, 0.274069, 0.451862]
(exp(-inf) = 0, so masked entries contribute nothing to the sum -- that is the whole trick)

STEP 5 -- weighted sum of V rows      [3,3] x [3,4] -> [3,4]
out[0] = 1.000000 * (1,2,3,4)
      = ( 1.000000, 2.000000, 3.000000, 4.000000)

out[1] = 0.377541*(1,2,3,4) + 0.622459*(5,6,7,8)
      comp 0: 0.377541*1 + 0.622459*5 = 0.377541 + 3.112295 = 3.489836
      = ( 3.489836, 4.489836, 5.489836, 6.489836)

out[2] = 0.274069*(1,2,3,4) + 0.274069*(5,6,7,8) + 0.451862*(9,10,11,12)
      comp 0: 0.274069 + 1.370345 + 4.066758 = 5.711172
      = ( 5.711172, 6.711172, 7.711172, 8.711172)

```

Read the output. Row 0 sees only itself and reproduces v row 0 exactly — the first token has no context and attention is a no-op for it. Rows 1 and 2 are convex combinations of the value rows they are allowed to see, so each output lands strictly inside the span of the visible values. Attention cannot invent information; it can only route and average what is already there. The MLP is what transforms.

1.5 Why $\sqrt{d}$ — the variance derivation

Treat the components of q and k as independent with mean 0 and variance 1 (which is roughly what normalisation plus standard initialisation gives you). Then for $q \cdot k = \sum_i q_i k_i$ over d terms:

```

E[q_i k_i] = E[q_i] E[k_i] = 0 (independent, both zero-mean)
Var(q_i k_i) = E[q_i^2 k_i^2] - 0 = E[q_i^2] E[k_i^2] = 1 * 1 = 1

E[q.k] = 0
Var(q.k) = sum over d independent terms = d <-- variance ADDS
SD(q.k) = sqrt(d)

```

So raw scores scale as $\sqrt{d}$. For this model $d_{\text{head}} = 128$, so $\sqrt{128} = 11.31$: before scaling, logits are routinely spread across ± 11 . Softmax is exponential, so that is catastrophic.

WORKED EXAMPLE 1-C – WHAT SATURATION DOES, NUMERICALLY

Take three scores one standard deviation apart at $d = 128$: $s = (+11.31, 0, -11.31)$

UNSCALED

```

exp(+11.31) = 81,633.9
exp( 0.00) = 1.0
exp(-11.31) = 0.00001225
sum          = 81,634.9
softmax      = (0.9999878, 0.0000122, 0.0000000015)

```

-> one token receives 99.999% of the weight. Attention has collapsed to a hard argmax. Gradients through the two suppressed entries are ~ 0 , so the layer cannot learn to change its mind. This is what "softmax saturates" means concretely.

SCALED by $1/\sqrt{128} = 0.08839$: $s' = (+0.99967, 0, -0.99967)$

```

exp(+0.99967) = 2.71739
exp( 0.00)    = 1.00000
exp(-0.99967) = 0.36800
sum           = 4.08539
softmax       = (0.66515, 0.24477, 0.09008)

```

-> a genuine soft distribution: a clear preference, but the other two still contribute and still receive gradient.

The division by $\sqrt{d}$ is exactly the normalisation that returns the score distribution to unit variance regardless of head width, so softmax behaves the same whether head_dim is 64 or 128. It is a variance-control argument, not a heuristic, and being able to derive it in three lines is the thing being tested when someone asks "why $\sqrt{d}$ and not d ?" — dividing by d would over-correct, crushing the scores toward uniform, because the *standard deviation* grows as $\sqrt{d}$ and it is the standard deviation that sets the scale.

1.6 Softmax numerical stability, and the seed of FlashAttention

$\text{softmax}(s)_i = e^{(s_i)} / \sum_j e^{(s_j)}$ overflows the moment any s_i is large. In fp16 the maximum finite value is **65,504**, and $e^{11.31} = 81,637$ already exceeds it. The fix uses the shift-invariance of softmax:

```

softmax(s)_i = e^{(s_i - m)} / sum_j e^{(s_j - m)}    for ANY constant m -- the m cancels

take m = max_j s_j, so every exponent is <= 0 and every e^{(...)} is in (0, 1]

```

WORKED EXAMPLE 1-D – MAX SUBTRACTION, AND THE CHECK THAT NOTHING CHANGED

```
s = (11.31, 0, -11.31), m = 11.31
```

```
exp(11.31 - 11.31) = exp( 0.00) = 1.0
exp( 0.00 - 11.31) = exp(-11.31) = 0.00001225
exp(-11.31 - 11.31) = exp(-22.62) = 0.00000000015
sum = 1.0000123
```

```
softmax = (0.9999878, 0.0000122, 0.00000000015)
```

Identical to the unscaled result in 1-C to every printed digit, but the largest intermediate is 1.0 instead of 81,634 -- so it survives fp16, where the direct form returns inf/inf = NaN.

FORWARD REFERENCE – THIS IS THE SEED OF FLASHATTENTION'S ONLINE SOFTMAX (WEEK 2, DAYS 2-4)

The max-subtraction trick is normally taught as overflow protection. It is more than that. Notice what it implies: **if you later discover a larger max, you can repair an already-computed partial softmax by rescaling it**. Given a running max m , a running denominator l and a running output accumulator o , a new block with max m' is folded in as:

```
m_new = max(m, m')
scale = exp(m - m_new)                correction for what is already accumulated
l_new = l * scale + (that block's sum, shifted by m_new)
o_new = o * scale + (that block's V contribution, shifted by m_new)
```

This makes softmax *incremental*, which is what lets attention be computed in tiles that fit in on-chip SRAM without ever materialising the full $N \times N$ score matrix in HBM. That is FlashAttention. The answer to "softmax needs the whole row for its denominator, so how can attention be tiled?" is this rescale factor, and it is the same $\exp(m - m')$ you already wrote down for stability. Sources: Milakov & Gimelshein, [arXiv:1805.02867](https://arxiv.org/abs/1805.02867); Dao et al., [arXiv:2205.14135](https://arxiv.org/abs/2205.14135).

1.7 Causal masking

An autoregressive model must not let position i see position $j > i$, or it would be trained on information unavailable at inference. Implemented by adding $-\infty$ to the disallowed score entries *before* softmax, because $e^{(-\infty)} = 0$, so those entries contribute exactly zero to both numerator and denominator. Adding a large negative constant rather than post-hoc zeroing is what keeps the remaining weights correctly renormalised: they still sum to 1 over the visible set, with no separate normalisation pass. In practice the mask is $-1e9$ or the dtype minimum rather than literal infinity, to avoid $\infty - \infty = \text{NaN}$.

At decode there is no mask. There is exactly one query, at the last position, and every cached key is by construction in its past. This is a small point with a large consequence: the decode attention kernel is a genuinely different shape from the prefill one ($1 \times n$ rather than $n \times n$), which is why engines ship separate kernels for the two and why FlashAttention's tiling has nothing to bite on during decode.

1.8 Multi-head attention

PREFILL, batch 1, 512 tokens	DECODE, batch 1, context n
x [1, 512, 1536]	x [1, 1, 1536]
q = x @ W_q [1, 512, 1536]	q = x @ W_q [1, 1, 1536]
view [1, 512, 12, 128]	view [1, 1, 12, 128]
transpose [1, 12, 512, 128]	transpose [1, 12, 1, 128]
k = x @ W_k [1, 512, 256]	k = x @ W_k [1, 1, 256]
view [1, 512, 2, 128]	view [1, 1, 2, 128]
transpose [1, 2, 512, 128]	transpose [1, 2, 1, 128]
(v identical to k)	append k,v to cache -> [1, 2, n+1, 128]
	expand 2 KV heads -> 12 (or a GQA-aware kernel)
scores q @ k^T [1, 12, 512, 512]	scores q @ k^T [1, 12, 1, n+1]
out s @ v [1, 12, 512, 128]	out s @ v [1, 12, 1, 128]
transpose+view [1, 512, 1536]	transpose+view [1, 1, 1536]
o_proj [1, 512, 1536]	o_proj [1, 1, 1536]

WORKED EXAMPLE 1-E – MULTI-HEAD IS FREE

Because $n_heads \times head_dim = d_model$ exactly ($12 \times 128 = 1536$):

parameters W_q is [1536, 1536] whether it is 1 head of 1536 or 12 heads of 128.
The head split is a RESHAPE of the same matrix. Identical count.

FLOPs, score computation, prefill P = 512:

1 head of 1536: $2 \times 1 \times 512 \times 512 \times 1536 = 805,306,368$
12 heads of 128: $2 \times 12 \times 512 \times 512 \times 128 = 805,306,368$ <-- identical

So: 12 independent relational patterns, at zero cost in parameters or arithmetic.

One head must express all of syntactic agreement, coreference and topical relevance in a single attention distribution. Twelve heads express twelve, and o_proj learns how to mix them. That is the representational argument, and it costs nothing.

The second-order benefit, which matters for Week 2. Smaller heads mean a smaller $\sqrt{d_head}$ (11.31 rather than 39.2 for a single 1536-wide head) and, more importantly, per-head score tiles small enough to fit in on-chip SRAM. FlashAttention's entire tiling strategy depends on head_dim being ~64–128 rather than ~1536. The architectural choice made for representational reasons in 2017 turned out to be the property the 2022 kernel needed.

1.9 RoPE, derived and verified numerically

The problem. Attention as defined so far is permutation-equivariant: permute the input tokens and the outputs permute identically. Nothing in $\text{softmax}(QK^T/\sqrt{d})V$ refers to position. "dog bites man" and "man bites dog" produce the same set of representations. Position must be injected explicitly.

Approach	Mechanism	Weakness
Learned absolute	a trainable <code>[max_pos, d_model]</code> table added to the embedding	Cannot extrapolate one position past training length — those rows were never trained. Also encodes absolute index when what matters linguistically is relative distance.
Sinusoidal absolute	fixed sin/cos of position added to the embedding	Defined at any position, so it extrapolates in principle, but the relative structure is only implicit and gets diluted as it propagates through 28 layers of transformation.
RoPE	<i>Rotates</i> q and k by an angle proportional to position, inside every attention layer	Still degrades far past the trained context, but the relative property is <i>exact</i> and holds at every layer. Extended by NTK/YaRN scaling.

The mechanism. Split `head_dim = 128` into 64 pairs of adjacent components. Pair `i` gets frequency $\theta_i = \text{base}^{(-2i/d)}$ with `base = 10,000`. For a token at position `m`, rotate pair `i` by angle $m \cdot \theta_i$:

```
(x_0, x_1) -> ( x_0 cos(m*theta) - x_1 sin(m*theta),
                x_0 sin(m*theta) + x_1 cos(m*theta) )

theta_0 = 10000^(0/128) = 1.0           fastest: one full turn every 2*pi ~ 6.3 positions
theta_63 = 10000^(-126/128) = 1.245e-4  slowest: one full turn every ~50,500 positions

Applied to q and k ONLY -- never to v, and never to the residual stream.
```

Applying it to q and k only is the design decision that makes it work: RoPE alters *similarity*, which is what positions should affect, without altering the *content* that gets averaged. And because it is a rotation, it is norm-preserving, so it cannot make any token's key systematically larger and therefore systematically more attended.

WORKED EXAMPLE 1-F — VERIFYING THE RELATIVE-POSITION PROPERTY NUMERICALLY

The claim: the dot product between a rotated query at position m and a rotated key at position n depends only on $m - n$. The algebra is one line — rotation matrices satisfy $R_m^T R_n = R_{n-m}$, so $(R_m q) \cdot (R_n k) = q^T R_m^T R_n k = q \cdot (R_{n-m} k)$ — but the numbers are what make it stick.

Take one pair, $i = 0$, so $\theta = 1.0$ rad per position.

q pair = (1, 0) k pair = (0, 1)

--- CASE A: $m = 5, n = 3$, offset $m - n = 2$ ---

rotate q by 5 rad: $\cos 5 = 0.283662$ $\sin 5 = -0.958924$

$q' = (1 \cdot 0.283662 - 0 \cdot (-0.958924), 1 \cdot (-0.958924) + 0 \cdot 0.283662)$
 $= (0.283662, -0.958924)$

rotate k by 3 rad: $\cos 3 = -0.989992$ $\sin 3 = 0.141120$

$k' = (0 \cdot (-0.989992) - 1 \cdot 0.141120, 0 \cdot 0.141120 + 1 \cdot (-0.989992))$
 $= (-0.141120, -0.989992)$

$q' \cdot k' = (0.283662)(-0.141120) + (-0.958924)(-0.989992)$
 $= -0.040030 + 0.949327$
 $= 0.909297$

--- CASE B: $m = 9, n = 7$, SAME offset 2, totally different absolute positions ---

rotate q by 9 rad: $\cos 9 = -0.911130$ $\sin 9 = 0.412118$

$q' = (-0.911130, 0.412118)$

rotate k by 7 rad: $\cos 7 = 0.753902$ $\sin 7 = 0.656987$

$k' = (0 \cdot 0.753902 - 1 \cdot 0.656987, 0 \cdot 0.656987 + 1 \cdot 0.753902)$
 $= (-0.656987, 0.753902)$

$q' \cdot k' = (-0.911130)(-0.656987) + (0.412118)(0.753902)$
 $= 0.598601 + 0.310697$
 $= 0.909297$

--- CROSS-CHECK against the closed form $q \cdot (R_{\{n-m\}} k)$, $n - m = -2$ ---

rotate k by -2 rad: $\cos(-2) = -0.416147$ $\sin(-2) = -0.909297$

$k'' = (0 \cdot (-0.416147) - 1 \cdot (-0.909297), 0 \cdot (-0.909297) + 1 \cdot (-0.416147))$
 $= (0.909297, -0.416147)$

$q \cdot k'' = (1)(0.909297) + (0)(-0.416147) = 0.909297$

CASE A 0.909297

CASE B 0.909297 all three agree to every printed digit

CLOSED 0.909297

What this demonstrates. Positions 5 and 3 give the same attention score as positions 9 and 7. The absolute indices are invisible to the dot product; only the gap of 2 survives. That is exact, holds at every layer, and is why RoPE is applied inside attention rather than once at the embedding. Note also that q and k keep their norms, so RoPE injects position without injecting magnitude.

Long-context extrapolation, in one paragraph. Beyond the trained context length the fast pairs have wrapped many times and the model has never seen those angle combinations, so quality falls off. *Position interpolation* divides positions by a factor so a longer sequence maps into the trained angular range, at the cost of resolution on the fast pairs. *NTK-aware scaling* instead raises the base (10,000 $\rightarrow$ larger), stretching slow pairs more than fast ones. *YaRN* combines both with a per-frequency ramp plus a temperature correction on the attention logits. Practical relevance here: this model's config declares `max_position_embeddings = 131,072`, which is what sized the 1.79 GB of legacy cos/sin buffers described in 1-A — and the OOM sweep reached 123,565 tokens, just under it. The context wall in this repo was memory, not RoPE.

1.10 MHA vs MQA vs GQA — the KV-cache arithmetic

```
KV bytes = 2 (K and V) x layers x kv_heads x head_dim x dtype_bytes x seq x batch
```

Note which head count appears: **kv_heads**, not query heads. Getting this wrong inflates every prediction by the GQA group size, which here is 6x. The formula is implemented as `kv_cache_bytes()` in `scripts/bench_common.py` with a comment saying exactly this.

WORKED EXAMPLE 1-G — PER-TOKEN KV COST UNDER ALL THREE SCHEMES, THIS MODEL

```
constants: layers = 28, head_dim = 128, fp16 = 2 bytes, batch = 1
```

```
MHA kv_heads = 12 (one per query head)
```

```
2 x 28 x 12 x 128 x 2 = 172,032 B/token = 168 KiB/token
```

```
GQA kv_heads = 2 (this model; 6 query heads share each) <-- ACTUAL
```

```
2 x 28 x 2 x 128 x 2 = 28,672 B/token = 28 KiB/token
```

```
MQA kv_heads = 1 (all 12 query heads share one)
```

```
2 x 28 x 1 x 128 x 2 = 14,336 B/token = 14 KiB/token
```

```
RATIOS
```

```
MHA / GQA = 172,032 / 28,672 = 6.0x exactly the group size, 12/2
```

```
GQA / MQA = 28,672 / 14,336 = 2.0x
```

```
WHAT 9,343 MiB OF FREE VRAM HOLDS (12,288 MiB card minus 2,945 MiB of weights)
```

```
9,796,845,568 bytes / 172,032 = 56,947 tokens under MHA
```

```
/ 28,672 = 341,686 tokens under GQA <-- 6x more context
```

```
/ 14,336 = 683,373 tokens under MQA
```

```
CROSS-CHECK against the actual measurement in this repo:
```

```
the OOM sweep died at 123,565 tokens, well under the 341,686 analytical ceiling,
```

```
because HuggingFace's torch.cat re-allocation makes the resident cost 2 x KV
```

```
(measured slope 62 KiB/token against the analytical 28) and fragmentation eats
```

```
the last stretch. Under MHA the same run would have died before ~28,000 tokens.
```

WORKED EXAMPLE 1-H – WHY GQA SHRINKS THE CACHE BUT NOT THE WEIGHTS

GQA changes exactly two matrices per layer, `k_proj` and `v_proj`. Everything else is untouched.

		MHA (12 KV heads)	GQA (2 KV heads)	saved
k_proj	1536 x (kv*128)	1536 x 1536	1536 x 256	
	+ bias	2,360,832	393,472	
v_proj	same	2,360,832	393,472	

	k+v per layer	4,721,664	786,944	3,934,720
	x 28 layers	132,206,592	22,034,432	110,172,160
q_proj	1536 x 1536	UNCHANGED -- 12 query heads either way		
o_proj	1536 x 1536	UNCHANGED -- it consumes 12 heads x 128 = 1536 either way		
MLP	41,287,680/layer	UNCHANGED -- GQA does not touch the MLP at all		
embed	233,373,696	UNCHANGED		
model total, MHA	=	1,543,714,304	+	110,172,160
model total, GQA	=		=	1,653,886,464
				1,543,714,304
WEIGHT reduction	=	110,172,160	/	1,653,886,464
KV CACHE reduction	=		=	6.66%
				83.3% (6x)

Say it in this form. GQA takes **6.7% off the weights and 83% off the KV cache**. The asymmetry is the whole point, and the reason is structural: the weights are dominated by the MLP (88% per layer) and by `q_proj` / `o_proj`, none of which GQA touches, while the KV cache is composed *entirely* of the K and V heads, which are precisely what GQA cuts. The number of query heads — and therefore the model's representational capacity in attention, its FLOPs, and its parameter count — is unchanged at 12. Only the number of distinct key/value *projections* falls, from 12 to 2, with each shared across a group of 6.

And the systems reason this is the right trade: weights are a one-time fixed cost paid once per forward pass and shared across the whole batch. KV cache is *per sequence* and *grows with context*, so at any realistic serving scale it is the term that decides how many concurrent requests fit. Cutting a constant by 6.7% is worth little. Cutting the term that scales with batch × context by 6× is what makes long-context serving possible at all.

1.11 A full decode-step trace, batch 1

One token, context length `n`. Shapes at every operation.

INPUT token id (a single integer), context length n already in the cache

```
1 embedding lookup          [1]                -> [1, 1, 1536]      row select; 0 FLOPs
                                                                    ~3 KB read from a 445 MiB table

--- repeat for each of 28 layers -----

2 input_layernorm (RMSNorm) [1, 1, 1536]        -> [1, 1, 1536]
3 q_proj                    [1,1,1536] x [1536,1536] -> [1, 1, 1536]  GEMV  4.72 MFLOP
4 k_proj                    [1,1,1536] x [1536, 256] -> [1, 1,  256]  GEMV  0.79 MFLOP
5 v_proj                    [1,1,1536] x [1536, 256] -> [1, 1,  256]  GEMV  0.79 MFLOP
6 reshape q                 [1, 1, 1536]        -> [1, 12, 1, 128]
6 reshape k, v              [1, 1,  256]        -> [1,  2, 1, 128]
7 RoPE on q and k, angle = n * theta_i          shapes unchanged
8 append k, v to the cache cache -> [1, 2, n+1, 128] each +28,672 B resident
9 expand KV 2 -> 12 heads (or a GQA-aware kernel reads with stride)
10 scores q @ k^T           [1,12,1,128] x [1,12,128,n+1] -> [1, 12, 1, n+1]
                                                                    no causal mask needed
11 softmax over the last dim [1, 12, 1, n+1]     -> [1, 12, 1, n+1]
12 out probs @ v            [1,12,1,n+1] x [1,12,n+1,128] -> [1, 12, 1, 128]
13 merge heads              [1, 12, 1, 128]      -> [1, 1, 1536]
14 o_proj                   [1,1,1536] x [1536,1536] -> [1, 1, 1536]  GEMV  4.72 MFLOP
15 residual add                                                     elementwise
16 post_attention_layernorm [1, 1, 1536]        -> [1, 1, 1536]
17 gate_proj                [1,1,1536] x [1536,8960] -> [1, 1, 8960]  GEMV 27.53 MFLOP
18 up_proj                  [1,1,1536] x [1536,8960] -> [1, 1, 8960]  GEMV 27.53 MFLOP
19 SiLU(gate) * up          [1, 1, 8960]        -> [1, 1, 8960]      elementwise
20 down_proj                [1,1,8960] x [8960,1536] -> [1, 1, 1536]  GEMV 27.53 MFLOP
21 residual add                                                     elementwise

--- end of layer loop -----

22 final RMSNorm            [1, 1, 1536]        -> [1, 1, 1536]
23 lm_head (tied embedding) [1,1,1536] x [1536,151936] -> [1, 1, 151936] GEMV 467 MFLOP
24 argmax (greedy)          [1, 1, 151936]      -> [1]                the next token id
```

EVERY WEIGHT MATRIX IS TOUCHED EXACTLY ONCE, AND EVERY ONE IS A GEMV (M = 1).

That single sentence is the whole reason decode is memory-bound.

1.12 Tying it to the roofline — the theoretical floor vs the measured rate

WORKED EXAMPLE 1-I — BYTES PER DECODE STEP, AND THE FLOOR THEY IMPLY

Use the profiling run's conditions: batch 1, context n = 525.

BYTES READ PER DECODE STEP

weights	1,543,714,304 params x 2 B	=	3,087,428,608 B
KV cache	28,672 B/token x 525 tokens	=	15,052,800 B
TOTAL		=	3,102,481,408 B = 3.102 GB

Activations add roughly another 0.15% on top and are ignored here.

Proportions of the traffic: weights 99.51%, KV cache 0.49%.

At batch 1 the KV cache barely matters for SPEED at this context. It matters for CAPACITY, and it matters for speed only once context or batch grows.

FLOOR = bytes / bandwidth

against ACHIEVABLE 291.5 GB/s: $3.102 / 291.5 = 10.64$ ms/token -> 94.0 tok/s

against THEORETICAL 360.0 GB/s: $3.102 / 360.0 = 8.62$ ms/token -> 116.1 tok/s

MEASURED (both from results/profile_summary.csv, same box, same session)

HuggingFace 40.65 ms/token = 24.6 tok/s -> 26% of the achievable floor

vLLM 12.56 ms/token = 79.6 tok/s -> 85% of the achievable floor

WHAT THE GAP IS MADE OF — AND THIS IS WHERE THE NEXT THREE WEEKS LIVE

HF excess over the floor	$40.65 - 10.64 = 30.01$ ms	
of which measured DEVICE IDLE	25.34 ms	= 84% of the excess
the remainder	4.67 ms	= kernels running below peak
vLLM excess over the floor	$12.56 - 10.64 = 1.92$ ms	
of which measured DEVICE IDLE	~0 ms	
all of it is kernels running at 84% of achievable rather than 100%		

The story the arithmetic tells. HuggingFace is not 3.2× slower than vLLM because its kernels are worse. It fires **1198 kernels per token with 1198 separate CPU launches**, and at 21.1 μs of idle per launch against an average kernel duration of 12.8 μs, the GPU spends **62.3% of every token doing nothing**. The dispatch chain above each kernel costs more than the kernel. vLLM's fused hand-written kernels cut the count to 356 (a 3.4× reduction) and CUDA-graph capture cuts the *launches* to 17, at which point the idle goes to zero and the step collapses to exactly the device work that was always there — predicted 13.02 ms, measured 12.57.

And the arithmetic closes the question of what comes next. vLLM at batch 1 is already at 85% of this card's achievable read bandwidth. There is essentially no idle left to remove, so no amount of further scheduling work can help. The only remaining lever is *moving fewer bytes per useful token*, which means raising arithmetic intensity above 2 — batching (measured: 30× throughput to batch 64), quantization (measured: 2.68× smaller weights, but 2.26× *slower* decode, because bitsandbytes dequantizes to fp16 before the GEMM so the matmul never sees a bandwidth saving), or speculative decoding. Every one of those is a Week 2 or Week 3 topic, and this single line of arithmetic — intensity 2 against a ridge of 142 — is why.

1.13 Self-test

Answer each unaided and out loud before reading on. Under 90 seconds each.

1. Why divide the attention scores by $\sqrt{d}$ rather than d ?
2. Multi-head attention with 12 heads of 128 costs how much more than 1 head of 1536, in parameters and in FLOPs?
3. Compute the KV cache for this model at 100,000 tokens, batch 1. Then say what it would be under MHA.
4. Why is RoPE applied to q and k but not to v ?
5. Softmax needs its whole row for the denominator. How can attention be computed in tiles?
6. Where does the "2 × params per token" FLOP rule come from, and when does it break?
7. Why is prefill 150× faster per token than decode on the same model and card?
8. At batch 1, what fraction of the bytes moved per decode step is the KV cache at context 525? Why does the answer make the KV cache important anyway?

Answers

1. **$\sqrt{d}$, because it is the standard deviation that grows as $\sqrt{d}$, not the variance.** With unit-variance independent components, $\text{Var}(q \cdot k) = d$ so $SD = \sqrt{d}$. Dividing by $\sqrt{d}$ returns the scores to unit variance. Dividing by d would over-correct, shrinking scores toward zero and flattening softmax toward uniform. At $d_{\text{head}} = 128$, unscaled scores spread ± 11.3 , which puts 99.999% of the softmax mass on one token (Worked example 1-C).
2. **Nothing, in either.** Because $12 \times 128 = 1536 = d_{\text{model}}$, the head split is a reshape of the same $[1536, 1536]$ matrix. Score FLOPs at 512 positions are 805,306,368 either way (Worked example 1-E). You get 12 independent relational patterns for free. The second-order benefit is that 128-wide heads produce score tiles small enough to fit in SRAM, which is what FlashAttention needs.
3. **$2 \times 28 \times 2 \times 128 \times 2 \times 100,000 = 2,867,200,000 \text{ B} = 2.67 \text{ GiB}$** (28 KiB/token × 100k). Under MHA with 12 KV heads it would be $6\times$ that, **16.02 GiB** — which does not fit on a 12 GB card at all, before even counting the 2.9 GB of weights. And in this repo's actual HF measurement the resident cost was $\sim 2.2\times$ the analytical figure because `torch.cat` keeps two copies live during the copy, so the real number at 100k was closer to 5.9 GiB.
4. **Because position should modulate *similarity*, not *content*.** RoPE's effect is entirely in the $q \cdot k$ dot product, where the rotation makes the score depend on $m - n$. The values are what gets averaged and returned into the residual stream; rotating them would corrupt the content a token contributes based on where it happens to sit. Rotation is also norm-preserving, so RoPE cannot make some position's key systematically larger and therefore systematically more attended.
5. **By carrying a running max and rescaling.** Keep m (running max), ℓ (running denominator) and o (running output). When a tile arrives with a larger max, multiply the accumulated ℓ and o by $\exp(m_{\text{old}} - m_{\text{new}})$ before adding the new tile's contribution. The result is *exact*, not approximate. That rescale factor is the same max-subtraction used for plain fp16 overflow safety (Worked example 1-D). It is what lets FlashAttention keep tiles in SRAM and never materialise the $N \times N$ score matrix in HBM — an IO result, not a FLOPs result.
6. **Each weight is touched once per token, and each touch is one multiply plus one add.** Verified exactly on `gate_proj`: 13,762,560 params, 27,525,120 FLOP/token, ratio 2.0000 (Worked example 0-D). It breaks in three places: it counts linear layers only and ignores attention's $O(\text{seq}^2)$ term ($\sim 3\%$ at 512 context, dominant past a few thousand); it ignores normalisation and elementwise ops, which are negligible in FLOPs but not in memory traffic; and it assumes every parameter participates, which fails for MoE.

7. **Operation shape, not arithmetic.** Prefill runs GEMMs with $M = 512$, arithmetic intensity ~ 512 FLOP/byte, which is right of this card's 142 ridge, so it is compute-bound and reaches $\sim 25\%$ of peak FLOPS. Decode runs GEMVs with $M = 1$, intensity ~ 2 FLOP/byte, $71\times$ left of the ridge, so it is bandwidth-bound. Both read the identical 3.09 GB of weights; prefill produces 512 tokens from that read and decode produces one. Measured here: 4205 tok/s vs 28.1 tok/s.
8. **0.49% — 15.05 MB of 3.10 GB.** At batch 1 and short context the KV cache is almost irrelevant to decode speed. It matters for two other reasons. First, **capacity**: it grows monotonically and is per-sequence, so it is what decides how many concurrent requests fit and how long a context can get — and it is what killed this card at 123,565 tokens. Second, **scaling**: weight bytes are shared across the batch and constant, while KV bytes scale with batch $\times$ context, so the KV read is the term that eventually catches the weight read. That crossover is exactly the batching knee this repo measures at batch 64, and it is why the knee did not move when the experiment was repeated on a $2.6\times$ -faster card.

1.14 Can you answer this? — the four most likely interview questions from this material

1. **Walk me through one decode step, layer by layer, with tensor shapes at batch 1.** → §1.11. Land on the closing sentence: every weight matrix is touched exactly once, and every one is a GEMV. Then give the consequence without being asked: intensity 2 against a ridge of 142.
2. **Why does GQA shrink the KV cache but not the weights?** → §1.10, Worked example 1-H. **6.7% off the weights, 83% off the cache.** Because the weights are dominated by the MLP and by q/o projections that GQA never touches, while the cache is composed entirely of the K and V heads it cuts from 12 to 2. Then say why the trade is right: weights are a constant shared across the batch, the cache scales with batch $\times$ context.
3. **Why RoPE rather than learned absolute positions?** → §1.9. Learned absolute cannot extrapolate past training length and encodes the wrong thing. RoPE's relative property is exact and holds at every layer, and it is norm-preserving. Then *show* it: positions (5,3) and (9,7) both give 0.909297 (Worked example 1-F). Close with one sentence on YaRN/NTK scaling.
4. **Softmax needs the whole row for its denominator, so how can attention ever be tiled?** → §1.6 and answer 5 above. The running max and the $\exp(m - m')$ rescale. Emphasise that it is exact, and that it is the same trick you already need for fp16 overflow safety. This is the bridge into Week 2, Days 2–4.

SOURCES

- Karpathy, [Let's build GPT from scratch, in code, spelled out](#) and the [nanoGPT](#) repo — §0.1–0.11, §1.2–1.8
- 3Blue1Brown, [Attention in transformers, visually explained](#) — §1.2–1.4
- Vaswani et al., *Attention Is All You Need* — [arXiv:1706.03762](#) — §1.4, §1.5 (the $\sqrt{d}$ footnote), §1.8
- Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding* — [arXiv:2104.09864](#) — §1.9
- EleutherAI, [Rotary Embeddings: A Relative Revolution](#) — §1.9, the clearest intuition for the pairwise rotation
- Ainslie et al., *GQA: Training Generalized Multi-Query Transformer Checkpoints* — [arXiv:2305.13245](#) — §1.10
- Shazeer, *Fast Transformer Decoding: One Write-Head is All You Need* (MQA) — [arXiv:1911.02150](#) — §1.10
- Zhang & Sennrich, *Root Mean Square Layer Normalization* — [arXiv:1910.07467](#) — §0.9
- Shazeer, *GLU Variants Improve Transformer* (SwiGLU) — [arXiv:2002.05202](#) — §0.8
- kippily, [Transformer Inference Arithmetic](#) — §0.7, §0.14, §1.12
- Pope et al., *Efficiently Scaling Transformer Inference* — [arXiv:2211.05102](#) — §0.14, §1.12; read twice before Week 1 Day 5
- Milakov & Gimelshein, *Online normalizer calculation for softmax* — [arXiv:1805.02867](#) — §1.6, forward reference to Week 2
- Dao et al., *FlashAttention* — [arXiv:2205.14135](#) — §1.6, §1.8 (why small heads matter)
- This repository: docs/baseline-hf-results.md, docs/oom-results.md, docs/profiling.md, docs/batching-results.md, scripts/bench_common.py, results/*.csv

Next unit: Week 1, Days 3–4 — GPU architecture, and the kernel gate in writing (`docs/gate-phase2.md`). See the companion document, *Three-Week Interview Roadmap*, §4.2.